

t-and-tup-talk-script

1. t and tup (title slide)

Thanks for having me. I'm going to show you what I've been building since late July. It's one repository with a few parts, but there is one question behind all of it, and I'll keep coming back to that question.

Since I first put this talk together, the project stopped being a plan and started running, and its purpose got sharp. The AI industry's bet is quantity: more data, more parameters, more compute, and clean the data well enough. I'm betting the opposite way: extreme data quality over quantity. Every single example a model learns from has to be proven correct by seven independent proof systems. locallm builds the models from scratch, and t is the filter that decides what they may learn from. I'll show why each part exists, what it measured this week, where it falls short, and how it stacks up against the industry.

The plan: the problem, the tool I built for it, the numbers it produces today, the filter running, and how it stacks up. About thirty minutes, then questions.

IF SOMEONE ASKS "What does t stand for?" Nothing. It's the letter t, the name of the language. tup is the name of the Linux distribution; the two names are the two halves of the repository.

2. The industry bets on quantity. This project bets on proof.

Here is the whole talk on one slide.

The industry's recipe for better AI is scale. Meta trained Llama 3 on more than 15 trillion tokens. The data is cleaned, carefully, but at that size cleaning has to be cheap: rules of thumb, removing duplicates, and other AI models scoring what looks like good text. Mistakes are expected, and the bet is that sheer volume washes them out.

I'm doing the opposite. Not trillions of examples, thousands. And every one of them is proven: seven independent proof systems must agree the program keeps its promise, a deliberately broken copy must be caught, the problem's tests must pass, and it can't be a copy of something already in the pool. Nothing gets in on a judge's opinion.

The models are small on purpose. locallm builds them from nothing on an ordinary computer, so the clean data is the only thing they ever learned. The question the project exists to answer: per parameter, does proven data beat piles of data? The first measurement says yes, at small scale, and I'll show it, including where it doesn't hold yet.

IF SOMEONE ASKS "Isn't this what Phi already did?" Phi moved in this direction, with filtered and synthetic textbook-quality data, and it worked. But it still decides quality with an AI's judgment, over billions of tokens. This goes to the extreme: no judge at all, only proof.

"Why would less data ever win?" Because a model spends its parameters learning whatever is in the data, including the mistakes. If nothing wrong is in the data, no parameters are spent on wrong things.

3. When a program says it is correct, how would you know?

This is the question behind everything I'll show you.

Say you have a program and someone claims it is correct. There are three ways to check that claim. You can run it once and see. You can test it on a bunch of inputs. Or you can prove it, meaning show that it does the right thing for every input, not just the ones you tried.

Here is why it matters more now than it did two years ago. A lot of code is now written by AI. It gets written fast, and the slow part is checking it. So the checker is the thing worth building, and the checker has to be trustworthy, because if the checker is wrong, every number that comes out of it is wrong.

That is how I got here. The first two months of this project were a training pipeline where a checker graded an AI's code. About half of the improvement I measured turned out to come from a bug in the checker, not from the model. I'll show that near the end. The rest of the project is what I built so that can't happen again.

IF SOMEONE ASKS "Isn't testing enough in practice?" Testing catches the inputs you thought of. Proofs cover the ones you didn't. For most software, tests are fine. For a checker that is grading an AI, and whose verdicts become training data, a wrong verdict poisons everything downstream, so I wanted the strongest check available.

4. A test checks one input. A proof checks all of them.

Let me make "prove" concrete, because this is the word the whole talk rests on.

On the left is a test. It says: when I call max with 3 and 5, I should get 5. That checks one input.

On the right is a specification. It says: for any x and y, the result is at least x, at least y, and is one of them. A proof shows that the code meets that statement for every possible x and y. Not by running them all, which is impossible, but by reasoning about the code the way you'd reason through a math proof.

The tools that check these proofs exist and have for a long time. Dafny from Microsoft Research, Lean and Rocq from the math community, SPARK for Ada and Frama-C for C in safety-critical industry, Verus for Rust, F Star from Microsoft and Inria. The catch: each has its own language, its own rules, and its own quirks. You write your program in their language, and they tell you "proved" or "not proved."

IF SOMEONE ASKS "How does the tool prove it without running every input?" It turns the code and the specification into logical formulas and hands them to a solver, or in Lean and Rocq's case checks a proof term step by step. If the formulas hold for all values, the program is proved. If the tool can't tell, it says so; that's called "unproved," and it is not the same as "wrong."

5. The parts, and why each one exists

Before the details, here is every part of the repository with the reason it exists, because each one was built to fix a specific problem the part before it exposed.

Every part serves one goal: data so clean that a small model built from it does not learn anything wrong. Quality over quantity only works if the quality is real, so most of the parts are there to make the filter impossible to fool. Models learn from data, so the data has to be clean. To call data clean, you need a check you trust. One proof system can be wrong, so there are seven, and they have to agree. A proof of a vague promise proves nothing, so every program gets a broken twin that the promise must catch. Seven proof systems each have their own language, so there is one language that translates into all seven. A filter nobody can rerun is just a claim, so every number sits in a file with the commit that made it, and tup exists so even the machine has receipts.

I'll come back to this table at the end with what each part measured.

IF SOMEONE ASKS "Why not just use tests as the filter?" Tests cover inputs someone thought of. This week showed both are needed: proofs catch what tests miss, and tests catch a proof of the wrong problem. The filter uses both.

"Isn't seven overkill?" Each proof system has had soundness bugs. Seven independent teams, seven different logics: a wrong answer has to fool all seven at once.

6. Why one verifier is not enough

So why not pick one of those tools and be done? Because a single "proved" can be wrong in three ways that the tool itself will never tell you about.

First, mistranslation. You have to write your program in the tool's language. If you translate it wrong, the tool proves a different program than the one you meant. It says "proved," and it is right, about the wrong thing.

Second, an empty specification. If the statement you asked it to prove is so weak that anything satisfies it, "proved" means nothing. The extreme case is "ensures true." Every program satisfies that. Less extreme cases are much harder to spot by eye.

Third, the quiet give-up. Tools time out, or run out of memory, or hit a case they can't handle. If whatever is wrapping the tool reports that as a pass, you get a green light that means nothing. This is the shape of the bug that bit my earlier pipeline: a component kept producing well-formed output after it had stopped measuring anything.

None of these are the verifier being unsound. They are all the human, or the script around the verifier, getting it wrong. So I wanted a setup where each of these would show up as a disagreement.

IF SOMEONE ASKS "Are the verifiers themselves ever wrong?" Rarely, and it's a big deal when they are. Lean and Rocq check every proof in a tiny trusted kernel, which is the strongest guarantee available. The others rely on a solver. My setup also has a way to catch a verifier bug: if a program that is known to be wrong gets "proved," that cell is flagged as unsound. It hasn't happened on the committed tasks.

7. One small program, seven independent verdicts

This is `t`. It's a small language I designed for one job: state what a function must do, and let seven verifiers check it.

Here is a real task from the repository. Read it top to bottom. "task max" takes two integers and returns one. The three "ensures" lines are the promises: the result is at least `x`, at least `y`, and is one of them. Then the body, which is ordinary code.

That one file is translated by a script into all seven verifiers' languages. Same program, seven translations. Each verifier checks its own translation and says proved or not. `t` itself proves nothing. Every verdict comes from a verifier with a long public track record.

Why seven? Go back to the three failure modes. If I mistranslated into one tool, the other six disagree. If the specification is empty, the next slide's trick catches it. If one tool quietly gave up, six others didn't. Seven verdicts on one program catch what one cannot.

One honest caveat: four of the seven use the same solver underneath, called Z3, so they are not seven fully independent opinions. But they are seven different translations and seven different front ends, and Lean and Rocq don't use Z3 at all.

IF SOMEONE ASKS "Why not just write directly in Dafny or Lean?" Then you have one verifier and all three failure modes. Also, `t` is much smaller than any of them, which matters for the AI part later: a small language is easier for a model to write and easier to check that it wrote it right.

"How big is t?" Integers, booleans, sequences, pairs, strings, loops with invariants, recursive helper functions, early return. No heap, no floats, no concurrency. Small on purpose.

8. Every task gets a twin, a deliberately broken copy

This is the answer to the empty specification problem.

Every task gets a twin. The twin is the same task with the body deliberately broken by a fixed rule. For max, the rule collapsed the if statement: the twin always returns x. The specification is untouched. Only the code is broken.

Then the rule is: a task counts only if two things are both true. The real program is proved in a verifier, and the twin is refuted in that same verifier. Refuted means the verifier accepted a proof that the specification fails at a specific input. Here that input is x equals 0, y equals 1: the real program gives 1, the twin gives 0, and 0 is less than y, so the promise breaks.

Why does this catch empty specifications? If a specification can't tell the real program from a broken one, it isn't saying anything. So a twin that gets proved is a red flag, and the task is rejected as vacuous.

Two details that took real work. First, the input that breaks the twin, called the witness, is found by running both programs in an interpreter before any verifier sees them. When I measured this, about nine percent of randomly generated twins behaved identically to the real program, so without the witness the whole test would be luck. Second, "refuted" is only ever minted from a verifier proving the failure. A verifier failing to prove the twin is not a refutation. That distinction cost me eleven rows on a table later, and I'll show it.

IF SOMEONE ASKS "Who picks how to break it?" Nobody. It's a fixed ladder of mutation rules applied the same way to every task, and the first rule that produces a behavior change with a witness is used. That way "the twin failed" always means the same thing.

"What does the verifier do with the witness?" The witness is turned into a small proof: at this input, the specification is false. The verifier checks that proof. That's what makes a refutation positive evidence, not the absence of a proof.

9. The seven verifiers

Here are the seven, with the versions I'm running and what each is built on. You don't need to know these tools. Three things are worth saying out loud.

They come from different communities. Dafny and F Star from Microsoft Research, Verus from the Rust world, SPARK from the Ada industry, Frama-C from the C safety world, Lean and Rocq from mathematics.

Four of them use the same solver underneath, Z3. Frama-C uses a different solver, Alt-Ergo. Lean and Rocq don't use a solver at all: they check a proof term in a small trusted kernel, which is the strongest kind of check there is.

All seven are installed without administrator rights, on Linux and macOS. Five run natively on Windows, all seven under the Windows Subsystem for Linux. There are install pages for each operating system in the repository, and the macOS and Windows ones were measured on real machines.

IF SOMEONE ASKS "Why these seven?" They are the mature, actively maintained, freely available program verifiers with a public record. If an eighth appears, the design lets me add a translation for it.

"Does everyone have to install all seven?" No. The tools run with whatever verifiers are present and refuse to give a verdict with fewer than two.

10. What agreement looks like

This is the table the whole project produces. One row per task, one column per verifier. Each cell has two verdicts: the real program, then the twin. Green means the real is verified and the twin is refuted. That's the only reading that counts.

Today the repository holds 34 hand-written tasks. 30 of them are green in all seven columns. The other four are blocked by named cells, and you can see two of them here. `min_max` times out in Rocq. `split_join` times out in SPARK, Frama-C abstains, and F Star can't prove it. Those are not failures of the program. They are a verifier not finishing, and the table says so instead of hiding it.

The second number is the conformance suite. Those are 66 probe tasks where I declared in advance what each verifier should say, including probes designed to be refuted, to be vacuous, or to be rejected. That suite tests my instrument, not the programs. It reads 457 of 462 cells right. The five open cells are all Frama-C, and each has a reason written down.

Every number in the repository is produced by a script and written to a file with a date on it.

IF SOMEONE ASKS "What does abstain mean?" The verifier declined to give a verdict, usually because the translation uses something it doesn't handle. It is recorded as abstain rather than as a pass or a fail.

"Is 30 of 34 good?" The four are known blockers with the verifier named. The table isn't the point; the point is that the table is honest and anyone can regenerate it.

11. Programs nobody wrote for t

Thirty-four tasks I wrote myself is a demo, not evidence. So the next step was programs nobody wrote for t.

DafnyBench is a public benchmark: 785 programs written in Dafny, one of the seven verifiers, with specifications. I built a translator, called the lifter, that reads a Dafny program and writes the t version. Every translation is checked two ways: a lemma that the verifier proves, and a differential run, meaning both programs are executed on the same inputs and must agree. Thirty-five semantic decisions the lifter had to make are written down, each with its reason.

The flow: 785 Dafny programs go in. 326 come out as t tasks; the rest use features t doesn't have yet, and the reason each was refused is recorded by name. Those 326 go through all seven verifiers with their twins. 182 of them read verified and refuted in all seven columns.

By a separate count, 334 of the 643 DafnyBench programs that can be graded fall inside t's current language. The gap between those numbers is lifter rules I still have to write.

IF SOMEONE ASKS "What's a lemma here?" A small statement the verifier proves that says the t translation and the original Dafny program compute the same thing. It's the verifier checking my translator.

"Why DafnyBench first and not Python?" Because Dafny programs already carry specifications. From Python you'd have to invent the specification, which is the AI part, later in the talk.

12. 57 of 164, and the bar is 82

Inside DafnyBench there is a subset of 164 programs that a language model wrote in Dafny, from problems in a public set called MBPP. Those 164 are the ones I care most about, because they are the shape of thing an AI writes: short functions from a one-sentence problem.

This chart is that number over the nine full sweeps since September 6. A sweep means: re-lift every program, re-run all seven verifiers on every task and every twin, and rewrite the table. It went 5, 34, 48, 53, 56, then down to 45, then 54, 56, 57.

Today: 99 of the 164 translate into t, and 57 of the 164 read verified and refuted in all seven verifiers. The bar I set for version 1.0 is 82, half of them, plus someone other than me reproducing the table from a clean checkout.

The drop from 56 to 45 is the next slide. It was on purpose.

IF SOMEONE ASKS "Why is 82 the bar?" Half. It's a round line that says the language covers the common case of what an AI writes, without claiming everything.

"What blocks the other rows?" The roadmap names each one. The biggest: a loop invariant about membership in a growing sequence that three verifiers can't yet prove, a size bound Frama-C needs from a loop invariant, and a bound lemma Rocq needs for recursive helper functions. After that it's translator rules for features like sets and real numbers.

13. Why the count went down once, on purpose

This one is about what a number is allowed to mean.

Until September 12, five of the seven verifier wrappers had a shortcut. If the verifier failed to prove the twin, the wrapper counted that as "refuted." It feels reasonable: the broken program didn't prove, so it's broken. But "I couldn't prove it" and "I proved it's wrong" are different claims. A timeout would count as a refutation. Incompleteness was being sold as evidence.

So I purged it. Now every verifier has exactly one door to "refuted": it must accept a proof that the specification fails at the witness input. Positive evidence only.

The cost was immediate: 56 of 164 dropped to 45. Eleven rows had been counting on the shortcut. Over the next three days I earned them back properly, by building the certificates the verifiers needed, and the count is now 57, above where it was, with every refutation meaning the same thing.

That is the rule the whole repository runs on. A count that rises because the rule got weaker is not progress.

IF SOMEONE ASKS "How did you find it?" Three adversarial audits of my own instruments after the first version went public. Each one was a session where the only goal was to break my own claims. The write-ups are in the repository as witness files with dates.

14. Auditing the instrument itself

If the checker is the product, the checker has to be attacked. Three things I did to my own tools.

First, fuzzing. I generated 1,009 random t tasks, ran all seven verifiers on each with their twins, 7,063 verdict cells, and looked for any error that all seven made together. None. Errors happen, but they don't line up, which is what seven independent checkers are for.

Second, the witness measurement from earlier. About nine percent of randomly generated twins behaved identically to the real program. That measurement is why every twin now has to carry a witness input before a verifier sees it.

Third, the audits found real holes. Frama-C's translation had a definedness gap, where an expression that is undefined in t was defined in C, fixed in the translation. And the refuted shortcut from the last slide. Both are written up as dated witness files.

Also in the repository: a conformance suite with declared expected verdicts, and a rule that when a verifier proves a twin that the interpreter says is wrong, that cell is flagged as unsound, the signal of a verifier bug. It has not fired on the committed tasks.

IF SOMEONE ASKS "What's a definedness gap?" In `t`, dividing by zero has no value; the program is undefined there. In `C` it does something. If the translation doesn't carry "undefined" across, `C` could prove something `t` never claimed. The fix makes the `C` version refuse the same inputs `t` does.

15. The corpus `t` is aimed at

Where is this going? At natural-language programming problems, the kind an AI is given every day: a sentence describing a function, and tests.

I collected 24,748 of them from four public sources, with their reference solutions and tests, checked byte for byte against a manifest. 4,239 of them are function-shaped; the rest read from standard input and need a signature extracted first.

Then I ran a census: for each problem, what would `t` need to express its solution, and which feature is the one thing blocking it. Today 772 of the 4,239 function-shaped problems fall inside `t`'s language, about eighteen percent. The census also ranks the missing features by how many problems each one unblocks. Unbounded loops, imports of libraries, tuples of three or more, and nested sequences top the list.

That ranking decides what gets added next. Each new feature gets a meaning in every one of the seven verifiers, and then the census runs again. In the two weeks before this talk that was: integer division, early return, sequences as values, break, sequence literals and slices, strings, pairs, nested sequences, and a seventeen-function string library. Each one lowered into all seven.

IF SOMEONE ASKS "Why not just support Python?" Every feature has to mean the same thing in seven verifiers, and I have to be able to check that it does. That's expensive per feature, so the census decides the order.

"Is eighteen percent low?" It's the honest number today, and there is a file that says exactly which feature moves it next.

16. The training loop

Now the AI part, which is where the project started and where it's going.

The loop: take a problem statement. A language model writes a `t` task for it: the signature, the promises, and the body. The seven verifiers grade that task, and the grader also runs the problem's own tests. Then the interesting move: the twin. If the model's task verifies and its twin is refuted, the refuted twin, with its witness input, is a concrete bug with a concrete counterexample. That pair, the good program and its broken sibling with the input that tells them apart, is the training data. The model trains on the bugs it can prove are bugs, and goes around again.

What makes this different from the usual "train on code that passes the tests" is the grader. Tests check inputs someone thought of. Seven verifiers plus a refuted twin check the specification for every input and check that the specification says something. The reward is much harder to fake.

The grader is one script, and it reproduces the whole agreement table cell for cell from the committed tasks, so the thing that grades the model is the same thing that produced every number in this talk.

IF SOMEONE ASKS "Which model?" So far a small open one, Qwen 2.5 Coder at 1.5 billion parameters, and a 7 billion one for a first experiment. Both run on the lab machine.

"Why is the twin the training signal and not the verified program?" The verified program alone tells the model what right looks like. The twin with a witness tells it exactly where wrong begins. That's the information tests don't carry.

17. What the loop measured with a small model

I ran the loop twice with a 1.5 billion parameter model, which is small; it fits on one graphics card. Held-out means problems the model never trained on, 161 of them.

The number that moved: answers that verified with a refuted twin in at least one verifier went from 9 to 15 after two rounds. With three samples per problem, 12 to 16. So the model got better at writing tasks the verifiers accept.

Two numbers didn't move, and they matter more. Test passes stayed flat, so it got better at satisfying the grader without getting better at the problems. And parse failures didn't move at all: at the start, 227 of 368 replies weren't even valid t, and after training, the same share. The model can't read the parser's error, so it can't fix it. I'm calling that the parse wall.

So the loop works mechanically, and it's paused. The next step needs a model large enough to read a verifier's error message and repair its answer. The script for that is written; what's missing is API access to a large model.

IF SOMEONE ASKS "Is 9 to 15 significant?" It's two rounds on a tiny model. It shows the pipeline works and that the twin signal moves the number the twins measure. I don't claim more than that, and the table with every stage is in the repository.

"Why not use a bigger open model on the lab machine?" The 7 billion one fits and was used for the first experiment. Larger ones don't fit alongside other people's jobs. The blocked step is a frontier-size model through an API.

18. tup is a Linux built from source, with a receipt for every step

The second half of the repository. tup is a Linux distribution built from source by a script, where every step leaves a receipt.

The base is Linux From Scratch, the book that walks you through building a Linux system by hand from source code. tup runs that book automatically. Every command is the book's own text. Every source archive is hashed before it's used. Every page records how long it took, whether it succeeded, and the hash of its own log. Any deviation from the book is a separate file that says what it changes and why. At the end there is a bundle of receipts anyone can audit.

The result boots. tup 0.1 boots from its own disk to a login prompt under an emulator, with nothing but firmware and the disk, in 20 to 45 seconds depending on the host. It was witnessed on macOS, Ubuntu, and Windows by someone other than me, and what "booted" means is one written definition: a login prompt appears, no kernel panic anywhere, the guest is still alive, and the waiting period elapsed.

I want to say plainly what tup is not. It is witnessed, not verified. It proves what was built, from which bytes, in what order. It proves nothing about the kernel or the C library being correct.

IF SOMEONE ASKS "Why build a Linux at all?" Next slide. Short version: the verifiers run on a machine, and a bug in the machine's software changed a verdict once.

"What architecture?" 64-bit ARM today. The x86 build is scripted and waits on a permission on the lab machine.

19. Why the machine is part of the project

Here's the July and August story I promised, and it's why tup exists.

Before t, I built a training pipeline: an 8 billion parameter model, trained with a method called DPO, where instead of human judges the reward came from unit tests. Everything was preregistered, meaning I wrote down what I'd measure before running it. Result: the trained model did not beat its baseline. Plus half a point, p of 0.51. A clean null.

Then I retrained from the same code and data on a second machine, three times, and got plus 9 to 11 points every time. Same inputs, different answer. The difference turned out to be the verifier. On one machine it ran under Python 3.11, on the other 3.12, and the two versions treat one thing differently: a function that annotates its types without importing the typing module. One version fails that, the other doesn't. The retrained models had learned to emit that one import line, which appears in none of the 918 training examples, and that alone accounted for 48 to 58 percent of the gain.

So roughly half of a highly significant result was the measuring instrument, not the model. That became the paper, and it is the reason for the rest of the project. It's why t has seven verifiers instead of one, and it's why tup exists: if the verdict depends on the machine, then the machine has to be accounted for too, down to which bytes built it.

IF SOMEONE ASKS "Was the other half real?" Yes: plus 3.7 to 5.6 points, still significant in every retrain, concentrated on two benchmark tasks. The paper says exactly that.

"Where's the paper?" It's in the repository under forge/docs, revision 15, with the full preregistration and every table regenerable from the run ledger.

20. locallm builds the models; t decides what they learn from

locallm is not a model. It's a model maker: it builds a model from random numbers on whatever computer you have, from whatever data you give it. No downloaded weights, no account, nothing leaves the machine. A 3 million parameter model trains in under a minute.

Why that matters for this project: when you build the model from nothing, you control every byte it ever learned from. A downloaded model has already read the internet, good and bad, and you can't take that out. With locallm, the training data is the only thing the model knows, so filtering the data actually filters the model.

That is how the pieces fit. Think of a water filter. locallm is the source: it can learn from anything. t is the filter: only programs that seven proof systems accept, whose twin is caught, that aren't copies, pass through. What comes out the other side is a model built only from clean water. The next slides show that loop running.

IF SOMEONE ASKS "Why not fine-tune an existing open model instead?" We do both. A small open model fine-tuned on the clean pool is the fast route to beating other small models. locallm is the route where nothing unfiltered was ever in the model at all.

"Can locallm learn things other than code?" Yes, logs, sensor data, records. For non-code data the plan is that t proves the validator, the program that decides what counts as clean for that data, so the filter itself is proven.

21. Seven verdicts beside the code

For anyone to use t, it has to feel like a language, not a research script. So there's an editor story.

There's a language server, which is the standard way editors talk to a language: it gives errors at the exact token, hover text, go to definition, and formatting. It's written in plain Python with no dependencies. On top of it there's a VS Code extension. And the thing that makes it t rather than any other language: the seven verdicts, for the real task and its twin, appear beside the code, with the witness input when a twin is refuted.

An unchanged task costs no verifier run, because verdicts are cached by the hash of the file, so editing is fast.

Status, honestly: it's built, and the walk-through that a second person is supposed to follow from a fresh checkout is written, but no second person has run it yet. That's one of the two things the 1.0 bar requires. A Visual Studio version is planned and needs a Windows machine to build.

IF SOMEONE ASKS "Does it need all seven verifiers installed?" No. Absent verifiers show as absent. The walk-through is measured with Dafny present and Verus deliberately absent, to show both.

22. The bar for 1.0, and what blocks it

What does done look like? Two halves, and both have to hold before I tag a 1.0. There are no dates on the roadmap, on purpose. Every item has a finish line a third person can check.

Coverage: at least 82 of the 164 model-written programs lift into t and read all seven, and someone other than me reproduces the table from a clean checkout. Today 57 and 99 respectively.

Usability: a fresh checkout opens a t file in VS Code on Linux and Windows, and in Visual Studio on Windows, with the editor behaviors from the last slide and the seven verdicts, and a second person has repeated the walk-through. Today it's built for VS Code, nobody else has run it, and Visual Studio isn't built.

The blockers for the coverage half are named, not vague. A loop invariant about membership in a growing sequence that Verus, Lean, and F Star don't yet prove. A size bound Frama-C needs from a loop invariant. A bound lemma Rocq needs for recursive helper functions. Then translator rules for function contracts, real numbers, sets, and the remaining array shapes. Each sweep after each fix says the new number.

IF SOMEONE ASKS "When?" I don't give dates on this project. I name the next hurdle and what it unblocks. The next hurdle is the membership invariant, and it unblocks the largest group of rows in three verifiers at once.

23. Eight weeks, 785 commits

The whole thing, in order, so you can see how the parts connect.

July 25: first commit. A training pipeline with execution-verified pairs, and the same day, locallm, the from-scratch model.

July 28: the verifier gets pinned into every dataset receipt, because a passing receipt should never certify a weakened checker.

August: the preregistered null, the replication that wasn't, the paper. August 31: t goes live, with two verifiers in the morning and six by night, and tup 0.1 boots on three hosts the same day. Also the day the repository went public for the first time.

September 2 is the pivot. The training-first plan was set aside and the instrument came first. The refuted purge happened that day. From September 4 to 11, nine language features, each lowered into all seven verifiers, in the order the census ranked them, and the lifter. September 9, the loop measured at 1.5B. September 10, a survey of the field and seven moves chosen from it. September 12 to 15, nine sweeps, 5 to 57.

September 15 and 16, a 27B open model on the lab GPUs writes t for 649 problems, and the filter loop runs for the first time: locallm builds models, t keeps what is proven, locallm rebuilds. September 16 and 17, the whole thing moves to a MacBook, where all seven verifiers reproduce the table exactly, and that rerun catches a bug in my copy check that inflated the first result. I corrected it in the repository the same night.

Every one of those dates is a commit, and every number in this talk is in a file that says which commit produced it.

IF SOMEONE ASKS "Why the pivot?" Because the July and August result said the grader was the problem. Training a model against a grader you don't trust is measuring noise. So the grader became the project.

"How much of this was AI-assisted?" A lot of the code and drafting, under my direction, and the paper says so in its disclosure. The design decisions, the audits, and every claim are mine, and the point of the whole repository is that you don't have to take anyone's word for a number.

24. Quality beat quantity: same model, filtered against raw

This is the first measurement of the idea, and I'll tell you how it went, including the part where I got it wrong.

The experiment. `locallm` builds two models with identical settings: 3.2 million parameters, the same training steps, the same seeds, and the same amount of training text, about 95 kilobytes each. The only difference is the water. One learns from raw output a 27B model wrote, right and wrong mixed together. The other learns only from programs it let through. Each model then writes 500 programs, and every program goes through the same filter: it has to parse, follow the rules, not be a copy of anything the model trained on, and be proven by all seven verifiers with its broken twin caught.

The result: the model built from clean data wrote 29 new clean programs. The model built from raw data wrote 1. Then the loop: take the clean programs, add them to the pool, rebuild the model from scratch, write again. Round one reached 31.

Now the honest part. The first number I recorded was 46 against 3. When the loop was rerun on the MacBook, that run exposed a bug in my copy check: it compared a version line, so programs that were exact copies of training data, written with a different version line, counted as new. Recounted with the fix, 46 against 3 became 29 against 1. The direction held; the effect is about a third smaller. The correction is committed with the recount, next to the original numbers.

And quantity: a third model got ten times as much raw data, 925 kilobytes. It wrote 0 clean programs. One caveat, stated plainly: it trained for the same number of steps, so it saw each example less often. More raw data did not help at this size.

Why this matters: it's the smallest possible version of the thesis. Same model, and filtered data wrote 29 proven programs where the same amount of raw data wrote 1 and ten times the raw data wrote 0.

IF SOMEONE ASKS "Isn't 29 of 500 tiny?" Yes, it's a 3 million parameter model that trained for under a minute on 95 KB. The comparison is what matters: same everything, 29 against 1.

"Are the 29 interesting programs?" Mostly short, loop-free, and close to something in the training set. That's what a model this small can do. It's why the held-out test on the next slide matters more.

"How do you know the copy check is right now?" It erases everything the verifiers ignore: the name, the version line and the gate. Then it compares the program exactly. Every round in the repository was recounted with it, and the recount is committed.

25. What it does not do yet: problems it has never seen

The number that actually matters for competing with anyone: give a model a programming problem it never trained on, and see whether its answer is right and proven.

We hold out 232 problems. No training set ever contains them, and that split never changes.

The 27B open model gets 12 of the 232 fully clean: its program passes the problem's tests and is proven by all seven verifiers. It also gets 7 that are proven but wrong: all seven verifiers accept them and they fail the tests. That's a model writing a

promise that matches its own wrong code, and the proof faithfully confirms the code keeps the wrong promise. It's exactly why the filter needs the tests too.

The model locallm built from the clean data scores 0. And it's instructive how it fails: 188 of its answers are proven by all seven verifiers, and 151 of them are exact copies of programs it trained on. It ignores the problem and recites something it knows is verified. The verifiers alone would have rewarded that, which is why "clean" means all three at once: tests pass, proven by all seven, and not a copy.

The reason it scores 0 is plain: the clean pool has only 47 worked problem examples. You can't learn to solve problems from 47 examples. So the next step is volume.

IF SOMEONE ASKS "So does the idea work or not?" On the question it was built to answer first, yes: same size, same data, filtered data builds a model that writes far more proven programs. On solving new problems, not yet, and I'm not going to claim it does.

"What's the plan to get volume?" Many sampled answers per problem from an open model on a home RTX 4080, graded on the lab's CPUs, keeping only answers that pass their tests, are new, and are proven. Then a small open model trained on that pool, a locallm model built from it, and Phi-4-mini as the target, all scored on the same 232.

26. The opposite of industry, set against its best work

The industry's default is quantity. Even the labs that moved toward quality still decide what's good cheaply, because at trillions of tokens it has to be cheap. Here is how the leading work decides what's good, and what changes when you go all the way to proof.

Microsoft's Phi models, starting with "Textbooks Are All You Need," showed small models trained on carefully filtered, textbook-quality data beat much larger ones. Their filter is a model's judgment: an AI grades the quality of the data. t replaces the judge with proofs. A judge can be fooled; seven proof systems agreeing, plus tests, are much harder to fool.

DeepSeek-R1 and reinforcement learning with verifiable rewards train on answers a program can check, for code usually tests. Tests cover the inputs someone thought of, and published work in 2026 shows models learn to game imperfect verifiers. t's reward is a promise proven for every input, with a twin that stops empty promises.

AlphaProof and DeepSeek-Prover use a real proof kernel, Lean, as the reward. That's the closest in spirit, but it's one kernel, and for math competition problems. t uses seven independent kernels, for ordinary programs.

AlphaVerus translates DafnyBench into Verus and filters with a model's critique. The Vericoding benchmark has 12,504 tasks across Dafny, Verus and Lean, but grades each task in one system. t requires the same program to be proven in all seven, and checks its own translator.

Where we're behind, honestly: scale. They train billions of parameters on trillions of words. t is a small language, and our held-out score is 0 today. The bet is that clean data is worth more per parameter, and the size-matched result is the first measurement of that bet.

IF SOMEONE ASKS "Why would a lab care?" Because the thing they're all fighting is reward hacking and noisy data, and a filter that must fool seven independent proof systems at once is the strongest defense anyone has built for code.

"What would convince you it's wrong?" If a small model trained on the clean pool does no better per parameter on held-out problems than the same model trained on the same amount of test-filtered data. That comparison is planned and preregistered as the next result.

27. Limits, then questions

Let me end with the limits, because the repository states them and I'd rather you hear them from me.

t is small on purpose. No heap, no floating point, no concurrency. Four of the seven translations aren't hardened against hostile input; the committed tasks don't exercise those holes, and the holes are listed rather than hidden. tup is witnessed, not verified, and it's ARM only today. The filter loop works at small scale, and on problems it has never seen the model locallm built still scores 0, because the clean pool is small. And no second person has yet reproduced the main table from a clean checkout, though a second machine has, which is half of what 1.0 means.

If you want to try it: install two or more of the verifiers, read the tutorial, run the two commands on the screen. The first regrades every task in every verifier it finds and rewrites the table. It refuses to give a verdict with fewer than two verifiers present.

The repository is private while its claims are moving, and it's licensed for research and education. If anyone here wants access, ask me. And if anyone wants to be the second person who reproduces the table, that would move the project more than anything I can do alone.

To close where I started: the industry makes AI better by adding more. I'm trying to make it better by letting in only what can be proven, and measuring whether that wins per parameter. So far, at small scale, it does. Questions.

IF SOMEONE ASKS GLOSSARY, in case a term comes up:

- task: a t program: signature, requires, ensures, loop invariants, body.
- verifier / kernel: one of the seven proof systems. "Kernel" is the repository's word for the same thing.
- lowering: t's translation into one verifier's language.
- twin: the deliberately broken copy of a task; the specification is untouched.
- witness: the concrete input on which the real program and the twin differ.
- certificate: the small proof, built from the witness, that the specification fails at that input.
- verified / refuted: a cell that counts. Real proved, twin proved wrong.
- unproved: the verifier could not prove it. Not evidence either way.
- abstain: the verifier declined to give a verdict on this shape of program.
- vacuous / decorative: a specification that cannot tell the real from the twin.
- unsound: a verifier proved a twin the interpreter says is wrong; a verifier bug signal.
- matrix: the agreement table over the 34 committed tasks.
- sweep: the same table over the lifted DafnyBench tasks, regenerated from scratch.
- lifter: the translator from Dafny programs into t.
- DafnyBench: 785 public Dafny programs with specifications. MBPP-DFY: the 164 of them a language model wrote from MBPP problems.
- MBPP, HumanEval, APPS, CodeContests: public sets of programming problems with tests.
- fragment: the set of features t has today. census: a count of which programs of a corpus fit inside it.
- conformance suite: 66 probe tasks with a declared expected verdict per verifier; tests the instrument.
- receipt: tup's record of one build step: command bytes, source hash, duration, exit status, log hash.
- DPO: direct preference optimization, a way to train a model from pairs of better and worse answers.
- preregistered: what will be measured, and what counts as success, written down before the run.